

Weakest Preconditions (continue)

(wp and wlp in general programs)

- We need to be careful when nondeterminism is considered, $M(S, \sigma)$ might contain more than one (possibly pseudo) state.
 - $\sigma \in wp(S, q)$ iff $M(S, \sigma) \models q$.
 - $\sigma \in wlp(S, q)$ iff $M(S, \sigma) - \perp \models q$.
 - $\sigma \notin wp(S, q)$ iff there exist some $\tau \in M(S, \sigma)$ such that $\tau \not\models q$.
 - $\sigma \notin wlp(S, q)$ iff there exist some $\tau \in M(S, \sigma)$ such that $\tau \neq \perp$ and $\tau \not\models q$.
- 1. Show the following property: $wp(S, q_1) \wedge wp(S, q_2) \Leftrightarrow wp(S, q_1 \wedge q_2)$.
 - If a state $\sigma \in wp(S, q_1) \wedge wp(S, q_2)$, then $\sigma \in wp(S, q_1)$ and $\sigma \in wp(S, q_2)$; then $M(S, \sigma) \models q_1$ and $M(S, \sigma) \models q_2$; thus $M(S, \sigma) \models q_1 \wedge q_2$, which implies $\sigma \in wp(S, q_1 \wedge q_2)$.
 - If a state $\sigma \in wp(S, q_1 \wedge q_2)$, then $M(S, \sigma) \models q_1 \wedge q_2$; thus $M(S, \sigma) \models q_1$ and $M(S, \sigma) \models q_2$, which implies $\sigma \in wp(S, q_1) \wedge wp(S, q_2)$.
 - Using a similar proof, we can also show the following property: $wlp(S, q_1) \wedge wlp(S, q_2) \Leftrightarrow wlp(S, q_1 \wedge q_2)$.
- 2. Is it true that $wp(S, q_1) \vee wp(S, q_2) \Leftrightarrow wp(S, q_1 \vee q_2)$?
First, let's show that: $wp(S, q_1) \vee wp(S, q_2) \Rightarrow wp(S, q_1 \vee q_2)$.
 - If a state $\sigma \in wp(S, q_1) \vee wp(S, q_2)$, then $\sigma \in wp(S, q_1)$ or $\sigma \in wp(S, q_2)$; then $M(S, \sigma) \models q_1$ or $M(S, \sigma) \models q_2$; thus $M(S, \sigma) \models q_1 \vee q_2$, which implies $\sigma \in wp(S, q_1 \vee q_2)$.

How about the inverse of this property? Is it true that " $wp(S, q_1) \vee wp(S, q_2) \Leftarrow wp(S, q_1 \vee q_2)$ " ?

- When $M(S, \sigma) = \{\tau\}$ ($M(S, \sigma)$ contains only one state):
If $\sigma \in wp(S, q_1 \vee q_2)$, then $\tau \models q_1 \vee q_2$, and then $\tau \models q_1$ or $\tau \models q_2$; then $\sigma \in wp(S, q_1)$ or $\sigma \in wp(S, q_2)$ which implies $\sigma \in wp(S, q_1) \vee wp(S, q_2)$.
- When $M(S, \sigma)$ contains more than one states, then the statement is not necessarily true:
As a counterexample, let $\{\tau_1, \tau_2\} \subseteq M(S, \sigma)$. When $M(S, \sigma) \models q_1 \vee q_2$, it is possible that $\tau_1 \models q_1$ and $\tau_2 \models q_2$. So, even if we can have $M(S, \sigma) \models q_1 \vee q_2$, but don't necessarily have $M(S, \sigma) \models q_1$ or $M(S, \sigma) \models q_2$.

To sum up, $wp(S, q_1) \vee wp(S, q_2) \Leftarrow wp(S, q_1 \vee q_2)$ is not necessarily true when $M(S, \sigma)$ contains more than one state. In other words, we can prove that $wp(S, q_1) \vee wp(S, q_2) \Leftarrow wp(S, q_1 \vee q_2)$ is true when S is deterministic.

Using a similar proof, we can also show the following property: $wlp(S, q_1) \vee wlp(S, q_2) \Rightarrow wlp(S, q_1 \vee q_2)$. But $wlp(S, q_1) \vee wlp(S, q_2) \Leftarrow wlp(S, q_1 \vee q_2)$ only holds when S is deterministic (or $M(S, \sigma)$ contains only one state).

Calculate *wlp* for Loop-Free Programs

- We start with the calculations of *wlp* in loop-free programs because: 1) if a program is loop-free and runtime-error-free, then $wlp \Leftrightarrow wp$ 2) if a program can create runtime errors, then we can add “error-avoiding information” to convert *wlp* to *wp*. 3) We will handle loops in the future (we actually cannot find *wp* or *wlp* for loops).
- The following algorithm takes S and q and calculates a predicate for $wlp(S, q)$. This calculation procedure is pure textual, we use $\equiv$ here.
 - $wlp(\text{skip}, q) \equiv q$.
 - **[Backward Assignment Rule]** $wlp(v := e, Q(v)) \equiv Q(e)$, where Q is a predicate function.
 - This operation that takes us from $Q(v)$ to $Q(e)$ is called **syntactic substitution**; we will study this carefully in the future.
 - For example, $wlp(x := x + 1, x \geq 2) \equiv x + 1 \geq 2$
 $wlp(y := x * x, y \geq 4) \equiv x * x \geq 4$
 - $wlp(S_1; S_2, q) \equiv wlp(S_1, wlp(S_2, q))$.
 - $wlp(\text{if } B \text{ then } S_1 \text{ else } S_2 \text{ fi}, q) \equiv (B \rightarrow wlp(S_1, q)) \wedge (\neg B \rightarrow wlp(S_2, q))$
 $\Leftrightarrow (B \wedge wlp(S_1, q)) \vee (\neg B \wedge wlp(S_2, q))$.
 - $wlp(\text{if } B_1 \rightarrow S_1 \square B_2 \rightarrow S_2 \text{ fi}, q) \equiv (B_1 \rightarrow wlp(S_1, q)) \wedge (B_2 \rightarrow wlp(S_2, q))$.
- 3. Calculate the following weakest liberal preconditions.
 - a. $wlp(x := x + 1, x \geq 0) \equiv x + 1 \geq 0 \Leftrightarrow x \geq -1$
 - b. $wlp(y := y + x; x := x + 1, x \geq 0) \equiv wlp(y := y + x, x + 1 \geq 0) \equiv x + 1 \geq 0$
 - c. $wlp(y := y + x; x := x + 1, x \geq y) \equiv wlp(y := y + x, x + 1 \geq y) \equiv x + 1 \geq y + x \Leftrightarrow y \leq 1$
 - d. $wlp(x := x + 1; y := y + x, x \geq y) \equiv wlp(x := x + 1, x \geq y + x) \equiv x + 1 \geq y + x + 1 \Leftrightarrow y \leq 0$
 - 3.c and 3.d show that $wlp(S_1; S_2, q)$ and $wlp(S_2; S_1, q)$ do not have to be semantically equal.
 - e. $wlp(\text{if } y \geq 0 \text{ then } x := y \text{ fi}, x \geq 0) \equiv wlp(\text{if } y \geq 0 \text{ then } x := y \text{ else skip fi}, x \geq 0)$
 $\equiv (y \geq 0 \rightarrow y \geq 0) \wedge (y < 0 \rightarrow x \geq 0)$
 $\Leftrightarrow T \wedge (y < 0 \rightarrow x \geq 0)$
 $\Leftrightarrow y < 0 \rightarrow x \geq 0$
 - f. $wlp(\text{if } y \geq 0 \rightarrow x := y \square x < 0 \rightarrow x := y + 1 \text{ fi}, x \geq 0) \equiv (y \geq 0 \rightarrow y \geq 0) \wedge (x < 0 \rightarrow y + 1 \geq 0)$
 $\Leftrightarrow x < 0 \rightarrow y \geq -1$

Avoid Runtime Error in Expressions and Predicates

- Runtime errors can appear while evaluating expressions (and predicates even though it is not “runtime”). To avoid such errors while calculating $\sigma(e)$ (or $\sigma(p)$), we define **domain predicate** $D(e)$ (or $D(p)$) such that “if $\sigma \models D(e)$ then $\sigma(e) \neq \perp_e$ ” (or “if $\sigma \models D(p)$ then $\sigma(p) \neq \perp$ ”)
 - For example, to avoid error in the evaluation of $b[b[k]]$, we can define $D(b[b[k]]) \equiv 0 \leq k < \text{size}(b) \wedge 0 \leq b[k] < \text{size}(b)$.
 - Here is another example, we can define $D(x/y + u/v) \equiv y \neq 0 \wedge v \neq 0$.
- The calculation of $D(e)$ can be pure textual as well. We define $D(e)$ as follows:
 - If e contains no array selection, no “/”, no “%”, no $\text{sqrt}()$, then $D(e) \equiv T$
 - $D(b[e]) \equiv D(e) \wedge 0 \leq e < \text{size}(b)$.
 - $D(e_1/e_2) \equiv D(e_1 \% e_2) \equiv D(e_1) \wedge D(e_2) \wedge e_2 \neq 0$.
 - $D(\text{sqrt}(e)) \equiv D(e) \wedge e \geq 0$.
 - $D(\text{op } e) \equiv D(e)$.
 - $D(e_1 \text{ op } e_2) \equiv D(e_1) \wedge D(e_2)$ for all binary operator op other than “/” and “%”.
 - $D(f(e_1, e_2, \dots, e_n)) \equiv D(e_1) \wedge D(e_2) \wedge \dots \wedge D(e_n)$ for $f()$ other than $\text{sqrt}()$.
 - $D(\text{if } B \text{ then } e_1 \text{ else } e_2 \text{ fi}) \equiv D(B) \wedge (B \rightarrow D(e_1)) \wedge (\neg B \rightarrow D(e_2))$.

4. Calculate domain predicate $D(e)$ for the following expressions.

- $$\begin{aligned}
 D(x > 0 \rightarrow \text{sqrt}(x) > 0) \\
 &\equiv D(x > 0) \wedge D(\text{sqrt}(x) > 0) \\
 &\equiv T \wedge (D(\text{sqrt}(x)) \wedge D(0)) \\
 &\Leftrightarrow x \geq 0
 \end{aligned}$$
- $$\begin{aligned}
 \text{Let } B &\equiv 0 \leq k < \text{size}(b). \\
 D(\text{if } B \text{ then } b[k] \text{ else } -1 \text{ fi}) \\
 &\equiv D(B) \wedge (B \rightarrow D(b[k])) \wedge (\neg B \rightarrow D(-1)) \\
 &\equiv T \wedge (B \rightarrow D(b[k])) \wedge (\neg B \rightarrow T) \\
 &\Leftrightarrow B \rightarrow D(b[k]) \\
 &\equiv 0 \leq k < \text{size}(b) \rightarrow D(k) \wedge 0 \leq k < \text{size}(b) \\
 &\Leftrightarrow T
 \end{aligned}$$

Avoid Runtime Error in Statements

- To avoid runtime errors in the execution of S , we can define domain predicate $D(S)$ that gives a sufficient condition that avoids runtime errors.
- The calculation of $D(S)$ is textual as well. Let us define $D(S)$ as follows:
 - $D(\text{skip}) \equiv T$
 - $D(v := e) \equiv D(e)$
 - $D(b[e_1] := e_2) \equiv D(b[e_1]) \wedge D(e_2)$
 - $D(S_1; S_2) \equiv D(S_1) \wedge \text{wp}(S_1, D(S_2))$

- $D(S_1)$ guarantees the execution of S_1 is error-free.
 - $D(S_2)$ guarantees the execution of S_2 is error-free, so $wp(S_1, D(S_2))$ guarantees the of S_2 is error-free before S_1 is executed.
- $D(\text{if } B \text{ then } S_1 \text{ else } S_2 \text{ fi}) \equiv D(B) \wedge (B \rightarrow D(S_1)) \wedge (\neg B \rightarrow D(S_2))$.
 - $D(\text{if } B_1 \rightarrow S_1 \square B_2 \rightarrow S_2 \text{ fi}) \equiv D(B_1 \vee B_2) \wedge (B_1 \vee B_2) \wedge (B_1 \rightarrow D(S_1)) \wedge (B_2 \rightarrow D(S_2))$.
 - $D(\text{while } B \text{ do } S \text{ od}) \equiv D(B) \wedge (B \rightarrow D(S))$.
 - $D(\text{do } B_1 \rightarrow S_1 \square B_2 \rightarrow S_2 \text{ od}) \equiv D(B_1 \vee B_2) \wedge (B_1 \rightarrow D(S_1)) \wedge (B_2 \rightarrow D(S_2))$.
 - Although we cannot calculate wp or wlp for a loop, but we can calculate its domain predicate. We will discuss how to avoid divergence in the future.

Calculate wp for Loop-Free Programs

- $wp(S, q) \equiv D(S) \wedge wlp(S, q) \wedge D(wlp(S, q))$